Simulação e Teste de Software
Aula 04 — Técnicas Caixa-Branca: Critérios de Cobertura Estrutural

Solução Completa de Exercícios

Aluno: Lucas Rebouças Silva
R.A.: 22.122.048-6

Professor: Luciano Rossi
Centro Universitário FEI
Ciência da Computação

2º Semestre de 2025

Índice

1 Exercício 1 — Caminhos Independentes **3**
1.1 Código . . . 3
1.2 GFC e V(G) . . . 3

2 Exercício 2 — Cobertura de Comandos e Ramos **4**
2.1 Código . . . 4

3 Exercício 3 — Cobertura de Condição **5**
3.1 Código . . . 5

4 Exercício 4 — Teste de Ciclo **6**
4.1 Código . . . 6

5 Exercício 5 — Teste de Ciclo Aninhado **7**
5.1 Código . . . 7

6 Exercício 6 — Teste Completo (Integrador) **8**
6.1 Código . . . 8

7 Exercício 7 — Fluxo de Dados **9**
7.1 Código . . . 9

8 Resumo e Conclusões **10**

1 Exercício 1 — Caminhos Independentes

1.1 Código

```python
def verificar(n):
    if n > 0:
        if n % 2 == 0:
            return "Par positivo"
        else:
            return "Impar positivo"
    elif n < 0:
        return "Negativo"
    else:
        return "Zero"
```

1.2 GFC e V(G)

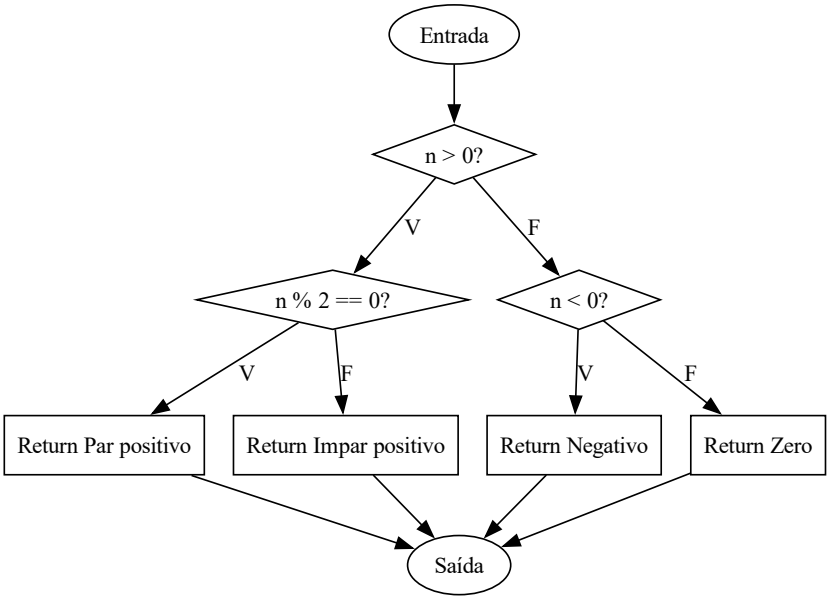

Figura 1: GFC Exercício 1

$V(G) = 3$. Caminhos: $n = 0$ (Zero), $n = 2$ (Par positivo), $n = -1$ (Negativo), $n = 3$ (Impar positivo). Casos de teste: (0, Zero), (2, Par positivo), (-1, Negativo), (3, Impar positivo).

2 Exercício 2 — Cobertura de Comandos e Ramos

2.1 Código

```python
def classificar(x):
    if x > 100:
        return "Alto"
    if x > 50:
        return "Medio"
    return "Baixo"
```

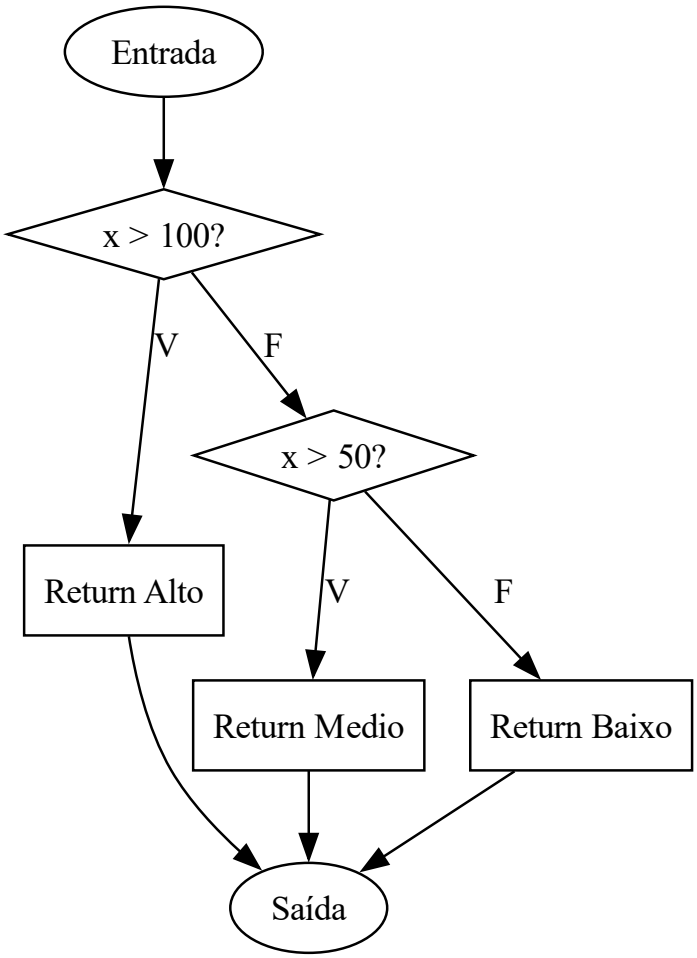

Figura 2: GFC Exercício 2

$V(G) = 3$. C0 e C1: 3 testes — (150, Alto), (75, Medio), (25, Baixo).

3 Exercício 3 — Cobertura de Condição

3.1 Código

```python
def acesso(idade, membro):
    if idade >= 18 and membro:
        return "Permitido"
    return "Negado"
```

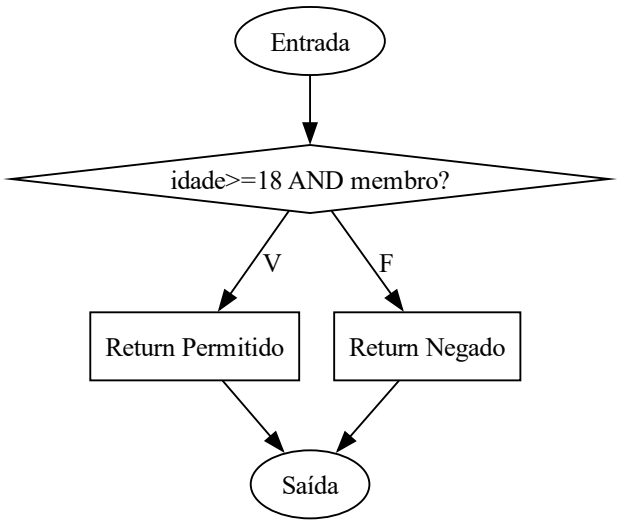

Figura 3: GFC Exercício 3

$V(G) = 2$. Condição: $(idade \geq 18) \wedge membro$. C1: 2 testes (25, True → Permitido; 20, False → Negado). CC: 4 testes para cobrir as 4 combinações (25/T, 25/F, 17/T, 17/F → Permitido só no primeiro).

4 Exercício 4 — Teste de Ciclo

4.1 Código

```python
def somar_ate(n):
    soma = 0
    for i in range(n):
        soma += i
    return soma
```

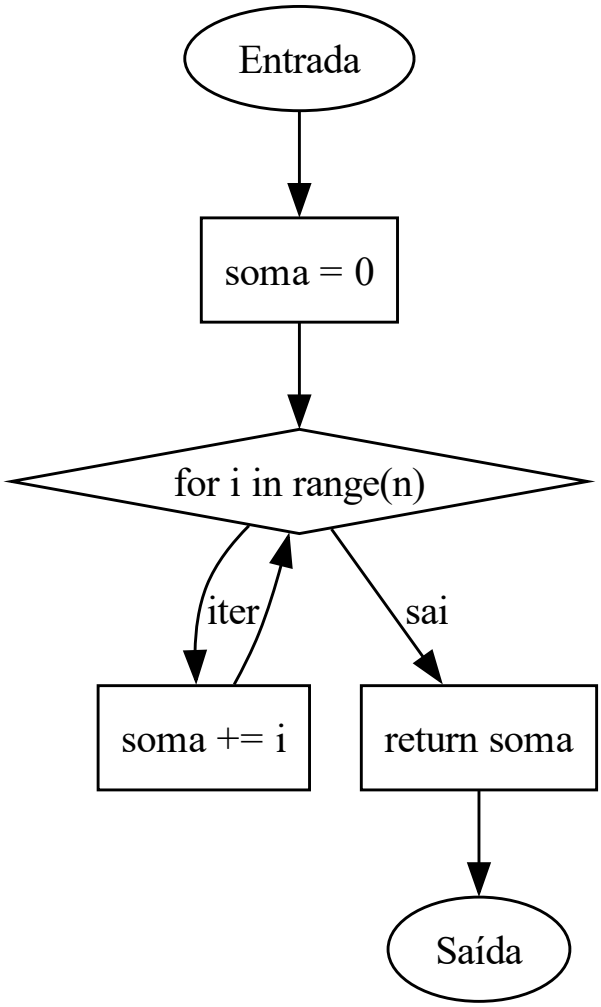

Figura 4: GFC Exercício 4

$V(G) = 2$. Teste de ciclo: 0, 1 e várias iterações. Casos: $n = 0 \to 0$, $n = 1 \to 0$, $n = 2 \to 1$, $n = 5 \to 10$, $n = 10 \to 45$.

5 Exercício 5 — Teste de Ciclo Aninhado

5.1 Código

```
def percorrer_matriz(m, n):
    posicoes = []
    for i in range(m):
        for j in range(n):
            posicoes.append((i, j))
    return posicoes
```

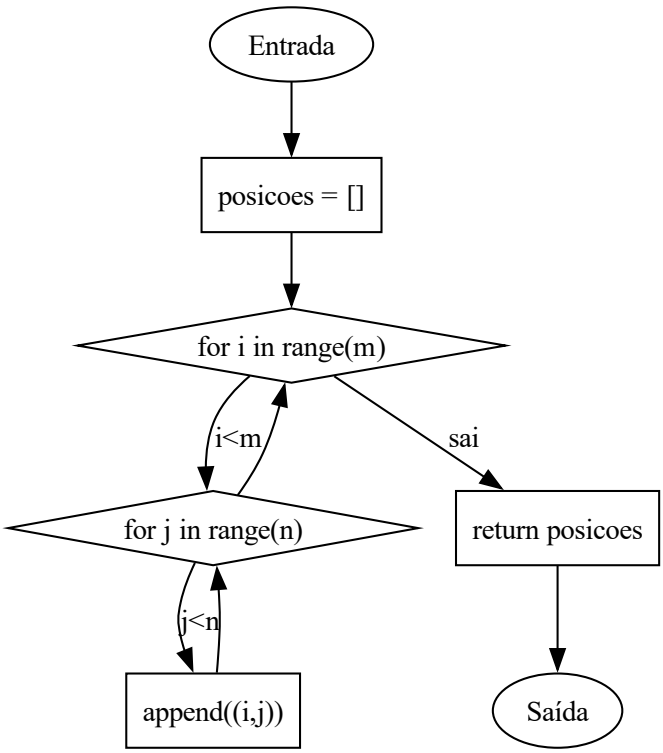

Figura 5: GFC Exercício 5

$V(G) = 3$. Iterações totais: $m \times n$. Cenários: (0,0)→0; (3,0),(0,3)→0; (1,3),(3,1)→3; (2,3)→6; (4,5)→20.

6 Exercício 6 — Teste Completo (Integrador)

6.1 Código

```python
def analisar(numeros):
    total = 0
    for n in numeros:
        if n > 0 and n % 2 == 0:
            total += n
        elif n < 0:
            total -= 1
        else:
            continue
    if total > 10:
        return "Acima"
    return "Abaixo"
```

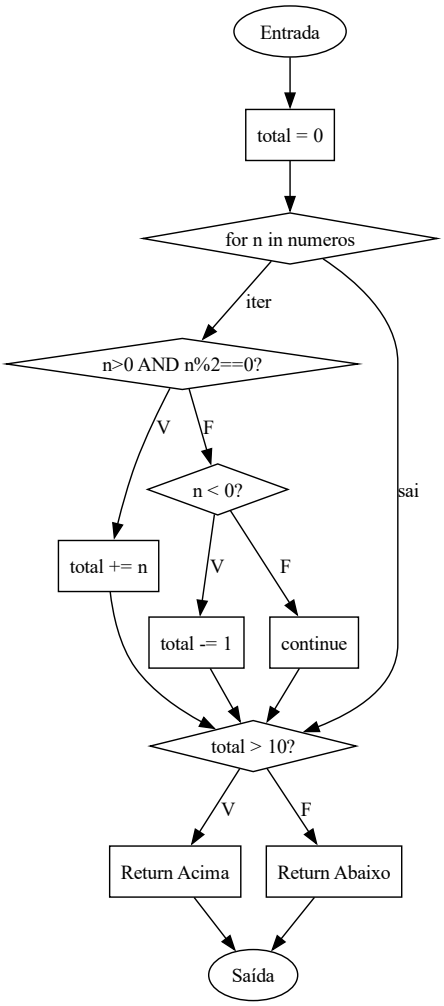

Figura 6: GFC Exercício 6

$V(G) = 5$. C0/C1: cobrir total+=n, total-=1, continue, return Acima/Abaixo e o desvio de laço (entrada/saída do `for`). Casos (resumo): []→Abaixo; [6]→Abaixo; [6,8]→Acima; [1],[-2]→Abaixo; [4,3,8],[6,-2,8],[0,6,8]→Acima; [2,4,1]→Abaixo. Def-uso `total`: def(2,5,7), use(10,11-12).

7 Exercício 7 — Fluxo de Dados

7.1 Código

```
1 def desconto(preco, cliente_vip):
2     total = preco                      # linha 2: def(total)
3     if cliente_vip:                    # linha 3: use(cliente_vip)
4         desconto_valor = preco * 0.2 # linha 4: def(desconto_valor)
5         total = preco - desconto_valor # linha 5: def(total)
6     if total < 50:                     # linha 6: use(total)
7         total = 50                     # linha 7: def(total)
8     return total                       # linha 8: use(total)
```

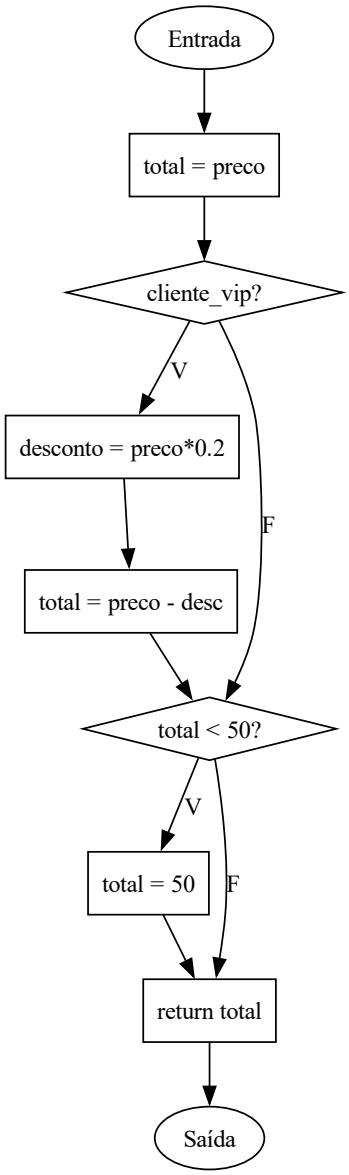

Figura 7: GFC Exercício 7 (def-uso)

$V(G) = 3$. As decisões são o teste de `cliente_vip` e o teste de `total < 50`, mais o caminho sequencial base. A análise de fluxo de dados considera as variáveis `total` (definições nas linhas 2, 5 e 7; usos nas linhas 6 e 8) e `desconto_valor` (definição na linha 4 e uso na linha 5). Uma suíte enxuta de testes que explora os caminhos com e sem cliente VIP, bem como valores acima e abaixo do piso de R$50, é suficiente para cobrir todos os pares def-uso relevantes (All-Defs e All-Uses).

8 Resumo e Conclusões

Hierarquia: C0 $\leq$ C1 $\leq$ CC $\leq$ caminhos; All-Defs/All-Uses complementam. Resumo: 7 exercícios, V(G) total 19, 39+ casos de teste. Recomendações: atingir no mínimo C1; usar CC em condições compostas; All-Uses em funções críticas.